Lecture 7

Modeling Schnapsen with Propositional Logic
Rule-based Reasoning
& Knowledge Graphs

Intelligent Systems
Vrije Universiteit

Overview

1 Propositional Logic for Schnapsen

2 Rule-Based Reasoning

3 Knowledge Graphs

- 3.1 Triple Syntax
- 3.2 Knowledge Engineering

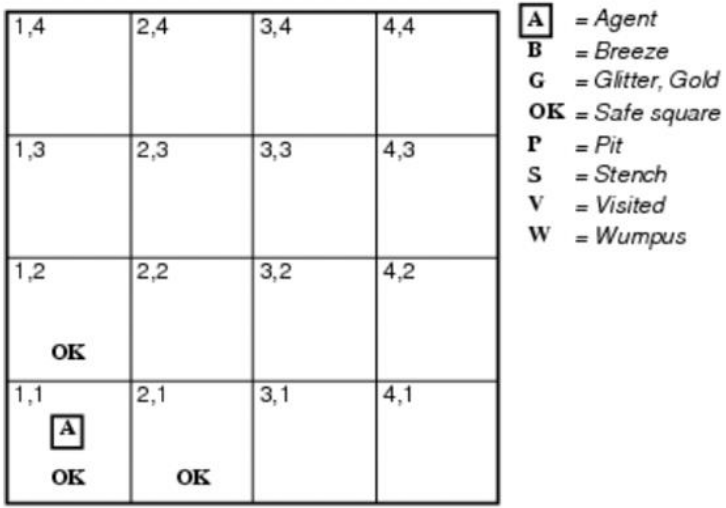
4 Tools

Part 1: Propositional Logic for Schnapsen

Intelligent Systems
Vrije Universiteit

Logic for Games

Recall propositional logic for Wumpus world.



(a)

Ways to use logic for games:

1. Reason over models or game states
2. Model strategies of the game

Reasoning over Models in Schnapsen

Phase 2: would it be safe to play King of Hearts?

Safe play in proposition logic: ¬ O10H ∧ ¬ OAsH ∧ (OQH ∨ OJH)

The opponent does not have a higher card, but a lower card of the same color.

(note: this does not work in Phase 1, as the suit does not have to be followed up)

Logical reasoning to deal with imperfect information:

- Up to now: strong assumption of perfect information in phase 2
- Human players usually forget which Trumps, Aces, and Tens have been played already.

Modeling Schnapsen Cards

	Aces	10s	Kings	Queens	Jacks
Clubs	0	1	2	3	4
Diamonds	5	6	7	8	9
Hearts	10	11	12	13	14
Spades	15	16	17	18	19

Encoding as propositions:

Card Values A0, A5, A10, A15, T1, T6, …, J14, and J19 are true, all others are false.

Color Values C0, C1, …, S18, S19 are true, all others are false.

Modeling Strategies

The goal of modeling a game strategy is to evaluate the properties of a card given the current card deck.

e.g. is the card entailed by the PlayNoTrumpJackStrategy?

Propositional Logic

PnTJS4 → J4 ∧ ¬ Trump4	J4 ∧ ¬ Trump4- → PnTJS4
PnTJS9 → J9 ∧ ¬ Trump9	J9 ∧ ¬ Trump9 → PnTJS9
PnTJS14 → J14 ∧ ¬ Trump14	J14 ∧ ¬ Trump14 → PnTJS14
PnTJS19 → J19 ∧ ¬ Trump19	J19 ∧ ¬ Trump19 → PnTJS19

First Order Logic

FOR ALL x: PnTJS(x) ⇔ J(x) ∧ ¬ Trump(x)

Modeling Strategies *First-Order Logic Example*

TrumpWeddingStrategy(x) ⇔ Q(x) ∧ Trump(x) ∧
THERE EXISTS y (samecolor(x,y) ∧ K(y) ∧ myCard(y)) (again, FOL)

samecolor(x,y) ⇔ (C(x) ∧ C(y)) v (D(x ∧ D(y)) ∨ (H(x) ∧ H(y)) v (S(x) ∧ S(y)
Samecolor12 ⇔ (C1 ∧ C2) v (D1 ∧ D2) v (H1 ∧ H2) v (S1 ∧ S2)
Samecolor13 ⇔ (C1 ∧ C3) v (D1 ∧ D3) v (H1 ∧ H3) v (S1 ∧ S3)
Samecolor14 ⇔ (C1 ∧ C4) v (D1 ∧ D4) v (H1 ∧ H4) v (S1 ∧ S4)
Samecolor15 ⇔ (C1 ∧ C5) v (D1 ∧ D5) v (H1 ∧ H5) v (S1 ∧ S5)
….

Mycard5, Mycard3, etc. as before.
K2, K7, K12, K17 as before (and all others false)
Q1, Q6, Q11, Q16 as before (and all others false)

TrumpWeddingStrategy0 ⇔ Q0 & Trump0 ∧ (
(samecolor01 ∧ K1 ∧ myCard1) ∨
(samecolor02 ∧ K2 ∧ myCard2) ∨
(samecolor03 ∧ K3 ∧ myCard3)
….)

Propositional Logic: Pros and Cons

PROs

- Declarative
- Compositional
- Allows partial/disjunctive/negated information, unlike most data structures
- Meaning is context-independent, unlike natural language

CONs

- Limited expressive power, unlike natural language

  E.g. “pits cause breeze in adjacent squares” can only be written using one sentence for each square

  → too complicated for Schnapsen

  → not meant to deal with partial observability

Part 2: Rule-Based Reasoning

Intelligent Systems
Vrije Universiteit

Inference Rules

An inference rule is a function which takes premises, analyzes their syntax, and returns a conclusion(s).
Rules are often abbreviated into short-form notation. Basic notation:

given A, we can derive B:

$$\frac{A}{B}$$

Note: this is not the same as A -> B (or A => B)

The implication is part of the object language to model things, the derivation is part of the meta language to manipulate symbols

Inference Rules

Modus Ponens

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

"And"-Elimination

from a conjunction any conjunct can be inferred

$$\frac{\alpha \wedge \beta}{\alpha}$$

Logical equivalences as rules

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$$

Logical Equivalence for Rewriting

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$
$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$
$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$
$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$
$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$
$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$
$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{de Morgan}$$
$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{de Morgan}$$
$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$
$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

Searching for Proofs

Monotonicity means the set of entailed sentences can only increase as information is added to the knowledge base.

$$\textit{for any sentence } a \textit{ and } b\textit{: if } KB \models a \textit{ then } KB \wedge b \models a$$

Finding proofs is alike to finding solutions to search problems.

Wumpus World Proofs *Example*

Assume R1 through R5 are true:

$$\neg P_{1,1}, B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}), B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1}), \neg B_{1,1}, B_{2,1}$$

How can we prove $\neg P_{1,2}$?

$R_6 : (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$	Biconditional elim.
$R_7 : (P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}$	And elim.
$R_8 : \neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1})$	Contraposition
$R_9 : \neg(P_{1,2} \vee P_{2,1})$	Modens ponens
$R_{10} : \neg P_{1,2} \wedge \neg P_{2,1}$	Morgan's rule

Alternative Rule-Based Reasoning: Forward/Backward Chaining

- Forward Chaining (FC) is data-driven, automatic, unconscious processing
  - Derive all the facts that can be derived from the KB by applying all the rules exhaustively
  - This set of facts is called the closure set (abbreviated KB*)
  - May do lots of work that is irrelevant to the goal
  - **KB $\vdash_f$ α : return α** $\in$ KB*

- Backward Chaining (BC) is goal-driven, appropriate for problem-solving.
  - Can we derive a specific fact from the KB?
  - Complexity of BC can be less than linear in size of KB

16

A New Calculus: Forward Reasoning

Remember:

- **KB ⊨ α**

  A sentence α is entailed by a KB K *iff* α is true in all models of KB.

- **KB $\vdash_f$ α:** Return **α** ∈ KB*

We can show soundness, completeness and termination of **KB $\vdash_f$ α** for Propostional Logic**.**

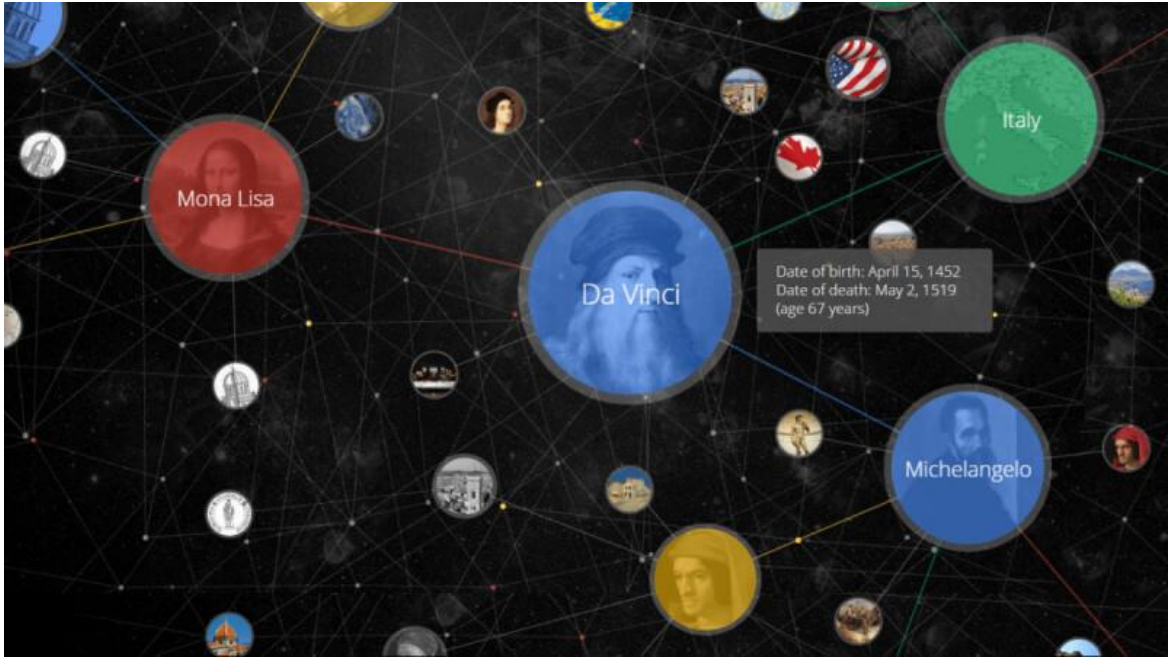
(this is because the closure KB* is finite for PL under some conditions).

Part 3: Knowledge Graphs

Intelligent Systems
Vrije Universiteit

Why do we need knowledge graphs?

AI and the Semantic Web requires specific knowledge that allows it to make associations between objects.



when was newton born - Google

https://www.google.nl/search?q=where+was+gone+with+the+wind+filmed&espv=210&es_sm...

Google

when was newton born

+Rinke Share

Web Images Videos Shopping News More Search tools

About 59,300,000 results (0.40 seconds)

January 4, 1643

Isaac Newton, Date of birth

Albert Einstein
March 14, 1879

Galileo Galilei
February 15, 1564

Johannes Kepler
December 27, 1571

Feedback

Isaac **Newton** - Wikipedia, the free encyclopedia
en.wikipedia.org/wiki/Isaac_**Newton**
by I Newton - Related articles
He was **born** three months after the death of his father, a prosperous farmer also named Isaac **Newton**. **Born** prematurely, he was a small child; his mother ...
Infinitesimal calculus - Principia - Newton's laws of motion - Nicolas Fatio de Duillier

Isaac **Newton** Biography - Facts, Birthday, Life Story - Biography.com
www.biography.com › People

Born on **January 4, 1643**, in Woolsthorpe, England, Isaac **Newton** was an established physicist and mathematician, and is credited as one of the great minds of ...

Isaac **Newton** Biography - **Newton's** Life, Career, Work - Dr Robert A...

Isaac Newton

Physicist

Sir Isaac Newton PRS MP was an English physicist and mathematician who is widely regarded as one of the most influential scientists of all time and as a key figure in the scientific revolution. Wikipedia

Born: January 4, 1643, Woolsthorpe-by-Colsterworth, United Kingdom

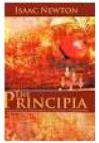
Died: March 31, 1727, Kensington, United Kingdom

Full name: Sir Isaac Newton

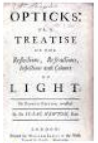
Education: Trinity College, Cambridge (1667–1668), more

Parents: Isaac Newton Sr., Hannah Ayscough

Books



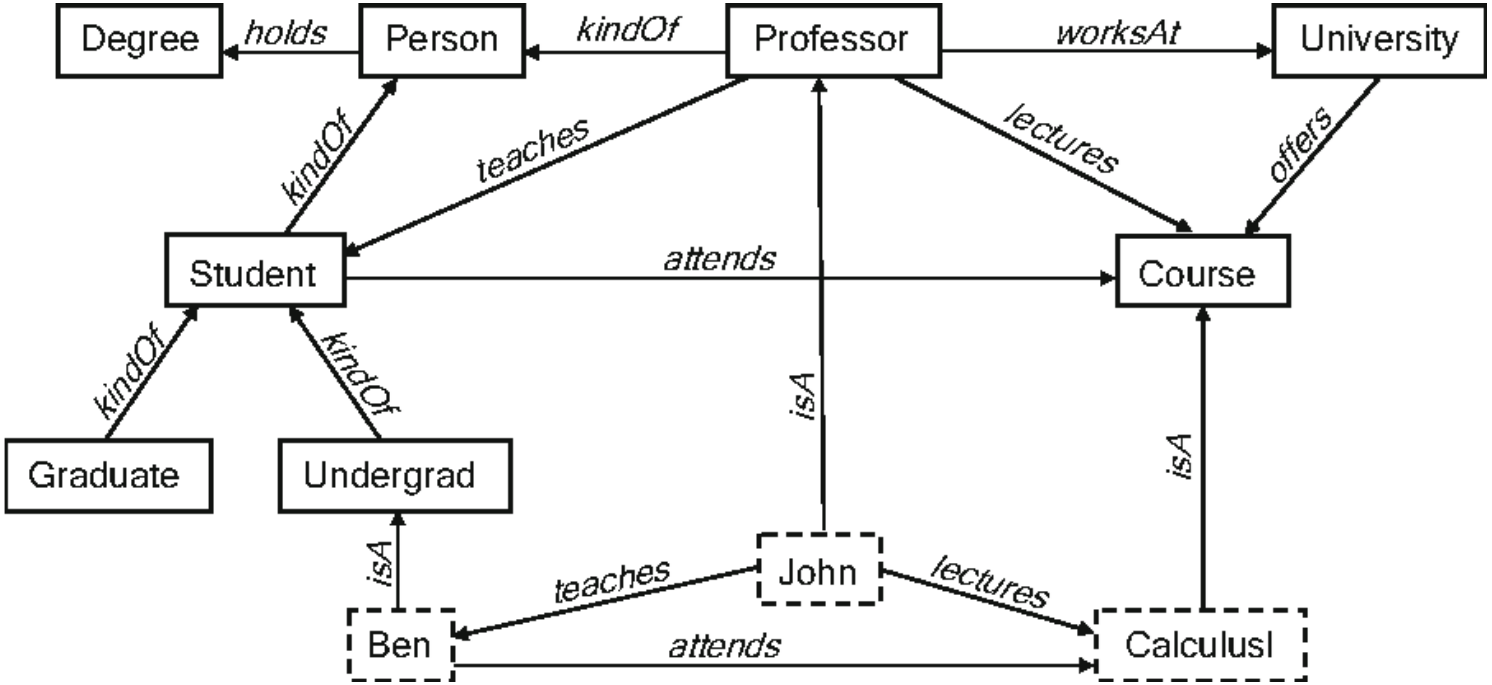
Philosophiæ Naturalis Principia...



Opticks

Method of Fluxions

What does it actually mean?



Grimm, Stephan. "Knowledge Representation and Ontologies." Scientific Data Mining and Knowledge Discovery (2010).

Ontologies

An ontology is an explicit specification of a *shared conceptualisation* that holds in a particular *context*.

Explicit	formal, machine interpretable language
Shared	consensus on definitions, shared openly on the Web
Conceptualisation	classes, instances, and relations
Context	tied to a particular domain or task

Components of Ontologies

1. Instances

Fido the dog

1. Classes

Dog

1. Attributes

fluffy, hasPaws

1. Relations

Fido **is a type of** Dog

Knowledge Representation: Formulas

Ontologies represent knowledge using logical formulas, such as

- Conditions on class membership, *e.g. all mammals are warm-blooded*
- Relations between classes, *e.g. all cities are populated places, every class is equivalent to itself*
- Assertions of class membership, *e.g. Amsterdam is a city*
- Characteristics of properties, *e.g.hasCapital only relates countries to cities, partOf is a transitive property*
- Assertions of property relations, *e.g. hasCapital(Netherlands,Amsterdam)*
- Assertions of equality, *e.g. morning star = evening star = venus*

Part 3.1: Triple Syntax

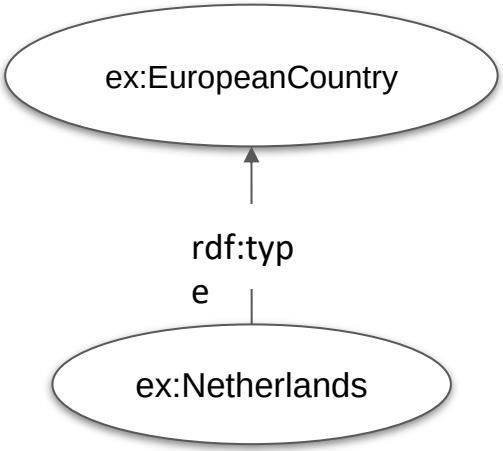
Basic Vocabulary: RDF for Knowledge Graphs

RDF(S) (Resource Description Framework) is a type of standardised vocabulary for modeling knowledge graphs.

e.g. to specify that something is a **member** of a **class**

`ex:Netherlands`	`rdf:type`	`ex:EuropeanCountry`
Instance	**Relation**	**Class**

Relations are like operators in propositional logic: they connect objects.



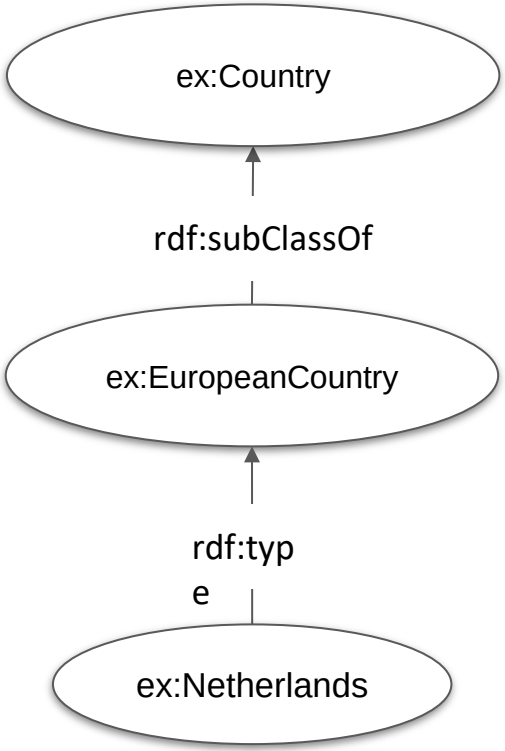
Basic Vocabulary: RDF Schema

e.g. to specify that something is a **subclass** of another **class**

```
ex:EuropeanCountry        rdfs:subClassOf   ex:Country

ex:Netherlands            rdf:type          ex:EuropeanCountry
```

```
⊨ ex:Netherlands          rdf:type          ex:Country
```



Basic Schema Vocabulary Summary

1. Specify that something is a **member** of a **class**

```
ex:Netherlands    rdf:type ex:EuropeanCountry
```

1. Specify that something is a **subclass** of another **class**

```
ex:EuropeanCountry        rdfs:subClassOf    ex:Country
```

1. Specify that some **property** always relates members of **specific classes**

```
ex:containsCity    rdfs:domain        ex:Country
ex:containsCity    rdfs:range         ex:City
```

1. **…and many more**

RDF Schema: Entailment Rules

1.	s	p	o	$\Longrightarrow$	s	rdf:type	rdfs:Resource
					p	rdf:type	rdf:Property
					o	rdf:type	rdfs:Resource
						OR	
					o	rdf:type	rdfs:Literal
2.	s	rdf:type	o	$\Longrightarrow$	o	rdf:type	rdfs:Class
3.	s	rdf:type	rdfs:Class	$\Longrightarrow$	s	rdfs:subClassOf	s
4.	p	rdf:type	rdf:Property	$\Longrightarrow$	p	rdfs:subPropertyOf	p

RDF Schema: Entailment Rules

5.	s	rdfs:subClassOf	y					
	y	rdfs:subClassOf	z	$\Longrightarrow$	s	rdfs:subClassOf	z

6.	p	rdfs:subPropertyOf	q				
	q	rdfs:subPropertyOf	r	$\Longrightarrow$	p	rdfs:subPropertyOf	r

7.	s	p	o				
	p	rdfs:subPropertyOf	q	$\Longrightarrow$	s	q	o

8.	s	p	o				
	p	rdfs:domain	x	$\Longrightarrow$	s	rdf:type	x

9.	s	p	o				
	p	rdfs:range	x	$\Longrightarrow$	o	rdf:type	x

10.	s	rdf:type	o				
	o	rdfs:subClassOf	t	$\Rightarrow$	s	rdf:type	t

Reasoning with Rules

We define: KG $\vdash_{rdf}$ (s, p, o) *iff* (s, p, o)

KG = {ex:EuropeanCountry rdfs:subClassOf ex:Country,
ex:BeneluxCountry rdfs:subClassOf ex:EuropeanCountry,
ex:Netherlands rdf:type
ex:BeneluxCountry}.

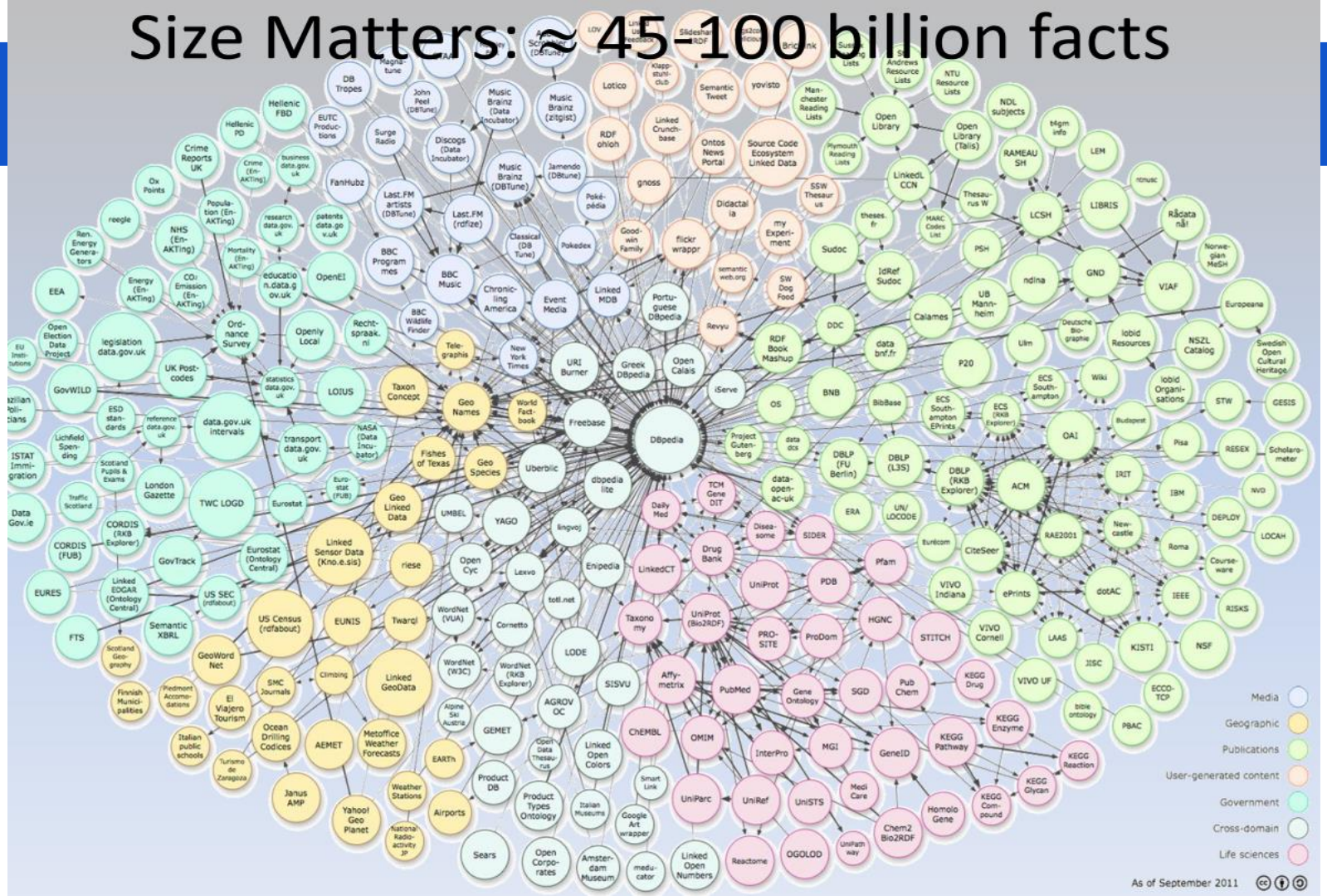
5.	s	rdfs:subClassOf	y					
	y	rdfs:subClassOf	z	⟹	s	rdfs:subClassOf	z

10.	s	rdf:type	o				
	o	rdfs:subClassOf	t	⇒	s	rdf:type	t

This means that our small KG also entails:

KG $\vdash_{rdf}$ ex:Netherlands rdf:type ex:Country

Size Matters: ≈ 45-100 billion facts



The Usual Rules for Logic Apply…

- Define semantics
  - Models are graphs, not truth assignments
- Show soundness and completeness of rules
- Devise and implement efficient reasoners
- Build applications on top of these knowledge bases (DBPedia, a variant of Wikipedia)

see Knowledge and Data in Year 2

Part 3.2: Knowledge Engineering

Intelligent Systems
Vrije Universiteit

What is Knowledge Engineering?

- Classes (concepts) and their hierarchy
- Properties (attributes & relations) and their hierarchy
- Class axioms (disjointness, equality)
- Class restrictions (universal, existential, cardinality)
- Property types (domain/range, functional, transitive)
- Instances (individuals)

Stages in Ontology Development

1. Determine domain & scope
2. Consider reuse
3. Enumerate terms
4. Define taxonomy
5. Define properties
6. Define classes and their properties
7. Define instances
8. Check for anomalies

Not a linear process!

Step 3: Enumerate Terms

Example for Schnapsen

trump

values

jack

cards

wedding

expensiveCard

king

tallon

suits

diamond

ranks

player1

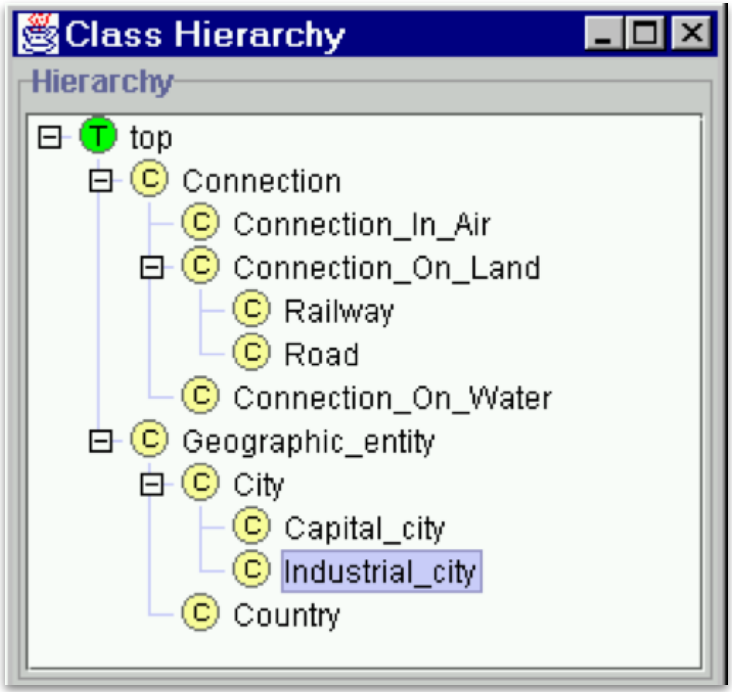
Step 4: Define Taxonomy (make sense of the terms)

Example triples for Schnapsen

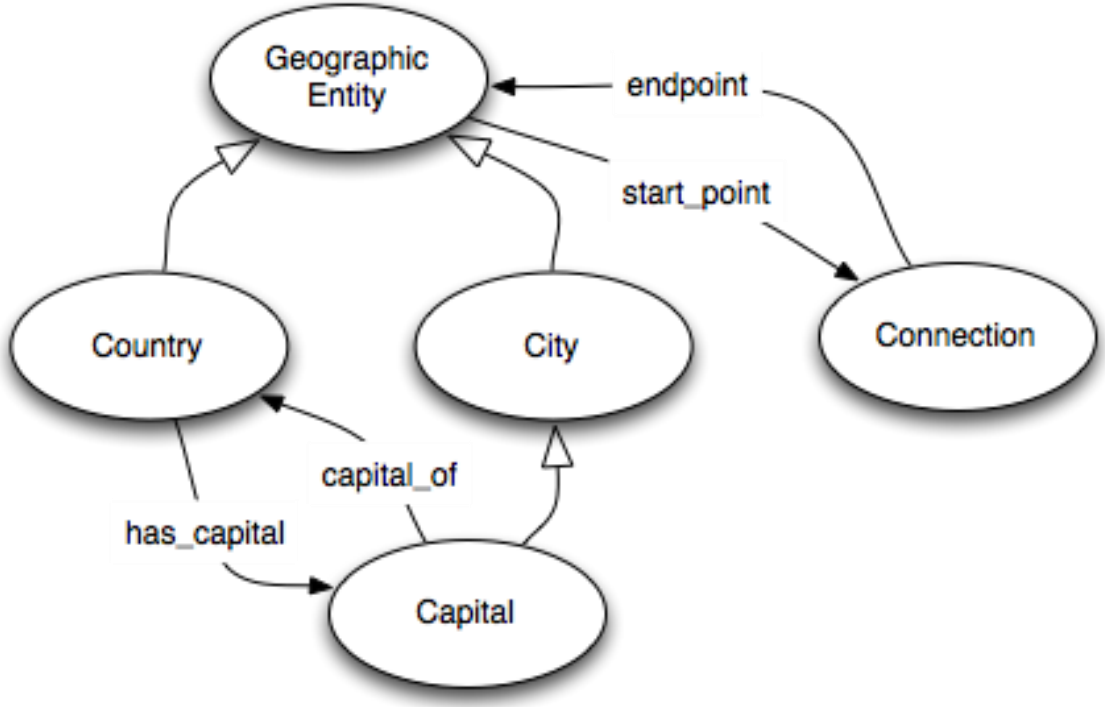
Jack is a Rank
subClass of a Class

expensiveCard is a Card
subClass of a Class

JackFourOfDiamond is type Card
Instance of a class



Step 5: Define Properties



Step 6: Define Classes and their Properties

Properties with global domain and/or range.

Domain is what a relation is applied to.

Range is the result of applying a relation.

e.g. the relation **played** has domain *player* and range *card*.

e.g. Emily **played** JackofSpadesFour

Does every instance of a class always have a certain datatype?

e.g. the relation **hasPoints** is always followed by an integer

Is a class or property disjoint with another class or property?

e.g. Emily can not be a card and a player, so card and player are disjoint

Step 7: Define Instances

Fill the ontology with specific instances.

e.g. given classes Player, Card, Jack and relation hasValue, we can add

JackOfDiamonds rdf:type Jack

Emily rdf:type Player

…and their properties

JackOfDiamonds disjoint JackOfHearts

JackOfHearts hasValue 2

Part 4: Tools

Tool Support: KGraphs in Practice

```
from rdflib import Graph, Literal, Namespace
from rdflib.namespace import RDF, RDFS

g = Graph()
s = Namespace("http://schnapsen.org/")

#suits
g.add((s.Hearts, RDFS.subClassOf, s.Card))
g.add((s.Spades, RDFS.subClassOf, s.Card))
g.add((s.Diamonds, RDFS.subClassOf, s.Card))
g.add((s.Clubs, RDFS.subClassOf, s.Card))
...

#additional information about the game
g.add((s.Jack, RDFS.subClassOf, s.CheapCard))
g.add((s.Ace, RDFS.subClassOf, s.ExpensiveCard))
```

SPARQL

SPARQL is a language that allows us to retrieve knowledge from a database by writing queries. We can use SPARQL inside of a Python environment.

```
result =  g.query("""
     PREFIX s:
     PREFIX rdf:
     SELECT ?card WHERE {
          ?card rdf:type s:player1
     }
     """)

for row in result:
    print('Result: {}'.format(row))
```

(SPARQL code is in green, Python in black.)

Knowledge Graphs with Entailment

```
import owlrl

rdfs = owlrl.RDFSClosure.RDFS_Semantics(g, False, False, False)
rdfs.closure()

result = g.query("""
            PREFIX s:
            SELECT ?card
            WHERE{
               ?card rdf:type s:Card .
            }
""")

for row in result:
   print('Result: {}'.format(row))
```

Intelligent Systems Framework

Efficient, simple, correct, well-understood, generic	Find best decision based on experience.	Weeks 6 & 7
	Find best decision with knowledge & reason.	Weeks 3, 4, 5
	Find best decision in multiple agent setting.	Weeks 1 & 2
	Find best decision with heuristics.	
	Systematically find all possible solutions.	

Summary

Take Home Message

We can do a lot with **propositional logic**, but it's sometimes very tedious or impossible (e.g. infinite domain).

There are different types of reasoning, like **rule-based** vs. **model searching**.

Knowledge graphs allow us to connect *instances* and *classes* with their properties by using *relations*. It's a type of rule-based reasoning scalable to billions of **triples** using tools like rdflib, SPARQL, and owlrl.